

Semana 6: Buenas prácticas de ingeniería — POO introductoria, Git, testing básico

Nivel: Básico-Intermedio | **Prerrequisitos:** Semana 5 (razonar sobre la corrección y eficiencia de una función antes de organizarla), Semana 4 (listas y diccionarios, usados como atributos), Semana 3 (funciones, `def`, `return`) | **Versión:** v1.0 | **Fecha:** 2026-09-28

Objetivos de aprendizaje

1. Explicar qué es la **Programación Orientada a Objetos (POO)** y crear clases básicas en Python usando `class`, `__init__` y `self`.
 2. Diferenciar **atributos de instancia** de **atributos de clase**, y explicar por qué usar una lista o diccionario como valor por defecto de un atributo de instancia es un error común.
 3. Usar los comandos esenciales de **Git** (`init`, `add`, `commit`, `status`, `log`) y un `.gitignore`, y explicar por qué el control de versiones es indispensable incluso en proyectos de una sola persona.
 4. Escribir **pruebas automatizadas** básicas con `pytest` (usando `assert` y `pytest.raises`), y explicar por qué las pruebas dan confianza para modificar código sin romperlo en silencio.
 5. Combinar las tres prácticas en un mismo proyecto pequeño: una clase con métodos, versionada con Git, cubierta por pruebas automatizadas.
-

1. De funciones sueltas a objetos: por qué organizar el código

Hasta la Semana 5, los datos y las funciones que los manipulan viven separados: por un lado una lista o un diccionario, por otro una función que recibe esos datos como parámetro. Funciona, pero a medida que un programa crece, esa separación empieza a pesar: hay que recordar qué funciones "van con" qué datos, y es fácil llamar a la función equivocada sobre los datos equivocados.

Analogía: un auto. Un auto tiene datos que le pertenecen (su color, la velocidad actual, el nivel de combustible) y comportamientos que actúan sobre esos datos (acelerar, frenar, cargar combustible). Nadie piensa en el auto como "una variable de color por un

lado y una función de frenar por otro, sin conexión entre ellas" — el color y el frenado son del **mismo** auto. La **Programación Orientada a Objetos (POO)** es exactamente esa idea aplicada a código: agrupar datos (**atributos**) y las funciones que operan sobre ellos (**métodos**) en una sola unidad, llamada **objeto**.

Esto no reemplaza nada de lo aprendido — las funciones, los `if`, los bucles, las listas y diccionarios siguen siendo las piezas con las que se construye todo. La POO es una forma de **organizar** esas piezas cuando los datos y su comportamiento pertenecen naturalmente juntos.

2. Clases y objetos en Python: `class`, `__init__`, `self`

Una **clase** es el molde que describe qué atributos y qué métodos va a tener cada objeto construido a partir de ella. Un **objeto** (o **instancia**) es el resultado concreto de usar ese molde.

```
class Estudiante:
    def __init__(self, nombre, notas):
        self.nombre = nombre    # atributo de instancia
        self.notas = notas      # atributo de instancia (una lista)

    def promedio(self):
        return sum(self.notas) / len(self.notas)    # misma lógica de la Semana 2, ahora como método

ana = Estudiante("Ana", [6.5, 7.0, 5.8])
luis = Estudiante("Luis", [4.2, 5.0, 6.1])

print(ana.promedio())    # 6.433333333333334
print(luis.promedio())   # 5.1
```

Tres piezas nuevas:

- **`class Estudiante:`** define el molde. Por convención, los nombres de clase empiezan con mayúscula.
- **`__init__`** es un método especial que Python llama automáticamente cada vez que se crea un objeto nuevo (`Estudiante("Ana", [...])`). Su trabajo es inicializar los atributos de esa instancia en particular.
- **`self`** es el primer parámetro de todo método de instancia, y representa "el objeto sobre el que se está trabajando en este momento". Cuando se escribe `ana.promedio()`,

Python pasa `ana` automáticamente como `self` — por eso `self` nunca se pasa explícitamente al llamar al método, solo al **definirlo**.

`ana` y `luis` son dos objetos distintos contruidos con el mismo molde: cada uno tiene **su propio** `self.nombre` y `self.notas`, completamente independientes entre sí.

3. Atributos de instancia vs. atributos de clase

Un **atributo de instancia** (como `self.nombre` arriba) pertenece a un objeto en particular. Un **atributo de clase** se define directamente dentro de la clase, fuera de `__init__`, y su valor es **compartido por todas las instancias**.

```
class Estudiante:
    escuela = "Aprende con Gutty IA"    # atributo de clase: mismo valor para todos los objetos
    total_estudiantes = 0                # también de clase: funciona como contador compartido

    def __init__(self, nombre, notas):
        self.nombre = nombre            # atributo de instancia: propio de cada objeto
        self.notas = notas
        Estudiante.total_estudiantes += 1  # se modifica sobre la clase, no sobre self

    def promedio(self):
        return sum(self.notas) / len(self.notas)

ana = Estudiante("Ana", [6.5, 7.0])
luis = Estudiante("Luis", [4.2, 5.0])

print(ana.escuela, luis.escuela)        # Aprende con Gutty IA / Aprende con Gutty IA (compartido)
print(Estudiante.total_estudiantes)     # 2 (se incrementó con cada Estudiante(...) creado)
```

Punto de confusión clásico: usar una **lista o diccionario mutable** como atributo de clase (por ejemplo `notas = []` en vez de `escuela = "..."`) es un error frecuente: como el atributo de clase es **uno solo, compartido**, todos los objetos terminarían modificando la **misma** lista. Cada objeto que necesite su propia lista de notas debe recibirla o crearla dentro de `__init__` (como `self.notas = notas`), nunca como atributo de clase.

4. Control de versiones con Git: por qué y comandos esenciales

Hasta ahora, si un archivo de código se rompe o se borra por accidente, la única forma de recuperar una versión anterior es si quedó una copia guardada a mano. **Git** es una herramienta que guarda el historial completo de cambios de un proyecto, para poder volver atrás, comparar versiones o entender **quién cambió qué y por qué** — y funciona igual de bien en un proyecto de una sola persona que en uno de cien.

Analogía: puntos de guardado de un videojuego. En un videojuego con puntos de guardado, si algo sale mal se puede volver al último punto guardado en vez de empezar todo el juego de nuevo. Un **commit** de Git es exactamente eso: una fotografía del proyecto completo en un momento dado, con un mensaje que describe qué cambió. A diferencia de "Ctrl+Z", el historial de Git no se pierde al cerrar el editor, y cada punto de guardado queda identificado y se puede volver a él en cualquier momento.

Comandos esenciales para empezar:

```
git init                # convierte la carpeta actual en un repositorio Git (una sola vez)
git status              # muestra qué archivos cambiaron desde el último commit
git add clase.py        # marca un archivo para incluirlo en el próximo commit
git add .               # marca TODOS los archivos modificados
git commit -m "Agrega clase Estudiante con método promedio()" # guarda el punto de guardado
git log                 # muestra el historial de commits, del más reciente al más antiguo
```

`git add` y `git commit` son dos pasos separados a propósito: `add` arma la lista de cambios que van a formar parte del próximo punto de guardado (el **área de preparación** o **staging area**), y `commit` recién los guarda de forma permanente en el historial, con un mensaje. Esto permite revisar (`git status`) exactamente qué se va a guardar antes de guardarlo.

Un archivo `.gitignore` le dice a Git qué archivos **no** debe rastrear nunca — típicamente archivos generados automáticamente (como la carpeta `__pycache__` que Python crea al ejecutar código) que no tiene sentido guardar en el historial porque se regeneran solos:

```
# .gitignore
__pycache__/
*.pyc
```

5. Testing automatizado: por qué probar el código con código

Hasta ahora, la forma de comprobar que una función funciona ha sido ejecutarla a mano y mirar el resultado con `print()`. Eso funciona una vez — pero cada vez que se modifica el código, hay que volver a probar todo a mano de nuevo, y es fácil olvidar revisar un caso que dejó de funcionar sin que nadie lo note. El **testing automatizado** es escribir código que verifica automáticamente que otro código se comporta como se espera.

La instrucción base de cualquier prueba es `assert`: si la condición que sigue es verdadera, no pasa nada; si es falsa, Python lanza un error inmediatamente.

```
def promedio(notas):
    return sum(notas) / len(notas)

assert promedio([6.0, 8.0]) == 7.0      # pasa en silencio: la condición es verdadera
assert promedio([10.0, 10.0]) == 10.0  # también pasa
```

Escribir `assert` sueltos funciona para probar algo rápido, pero en un proyecto real se usa una **biblioteca de testing** que organiza las pruebas, las ejecuta todas de una vez y reporta claramente cuáles pasaron y cuáles fallaron. La más usada en Python hoy es **pytest**: cualquier función cuyo nombre empiece con `test_`, dentro de un archivo que empiece con `test_`, es reconocida automáticamente como una prueba.

```
# test_promedio.py
from calculadora import promedio

def test_promedio_de_dos_numeros():
    assert promedio([6.0, 8.0]) == 7.0

def test_promedio_con_un_solo_numero():
    assert promedio([5.0]) == 5.0
```

Con esos archivos, ejecutar `pytest` en la terminal corre todas las funciones `test_*` automáticamente y muestra cuántas pasaron y cuántas fallaron — sin que haya que llamarlas ni imprimir nada a mano.

6. Por qué esto importa para IA/ML

Estas tres prácticas dejan de ser "buenas costumbres" y pasan a ser indispensables en cuanto un proyecto de IA/ML crece:

- La biblioteca de ML más influyente en la práctica, **scikit-learn** (Módulo 10), está construida sobre el mismo patrón de POO de esta semana: cada modelo es un objeto (por ejemplo un clasificador) con atributos (sus parámetros aprendidos) y métodos como `.fit()` (entrenar) y `.predict()` (predecir) — entender clases, `self` y métodos hoy es directamente entender cómo se usan esas herramientas más adelante.
- Un modelo entrenado sin control de versiones es casi imposible de reproducir: si el código cambió después de entrenar un modelo, puede volverse imposible saber exactamente qué versión del código produjo esos resultados. Por eso los equipos de ML versionan el código con Git desde el primer día, y existen herramientas especializadas (como DVC) que extienden la misma idea de Git al versionado de datasets y modelos.
- Los pipelines de datos y de entrenamiento son código como cualquier otro, y también se rompen en silencio: una función que limpia datos mal después de un cambio puede arruinar un entrenamiento completo sin lanzar ningún error visible. Cubrir con pruebas automatizadas las funciones de preparación de datos (por ejemplo, "esta función de normalización nunca debe devolver un valor mayor a 1") es lo que permite detectar ese tipo de error antes de gastar horas de entrenamiento sobre datos incorrectos.

7. Ejemplo práctico: una clase `CuentaBancaria`, versionada y probada

Problema: modelar una cuenta bancaria simple con depósitos y retiros, evitando que un retiro deje el saldo negativo, y verificar ese comportamiento con pruebas automatizadas.

Pseudocódigo:

CLASE CuentaBancaria

ATRIBUTO DE CLASE banco = "Banco Gutty"

METODO __init__(titular, saldo_inicial)

self.titular = titular

self.saldo = saldo_inicial

METODO depositar(monto)

SI monto <= 0 ENTONCES

LANZAR ERROR "el monto debe ser positivo"

self.saldo = self.saldo + monto

METODO retirar(monto)

SI monto <= 0 ENTONCES

LANZAR ERROR "el monto debe ser positivo"

SI monto > self.saldo ENTONCES

LANZAR ERROR "fondos insuficientes"

self.saldo = self.saldo - monto

Del pseudocódigo al código real (Python)

```
# cuenta.py
class CuentaBancaria:
    banco = "Banco Gutty" # atributo de clase: mismo banco para todas las cuentas

    def __init__(self, titular, saldo_inicial=0):
        self.titular = titular
        self.saldo = saldo_inicial

    def depositar(self, monto):
        if monto <= 0:
            raise ValueError("el monto a depositar debe ser positivo")
        self.saldo += monto

    def retirar(self, monto):
        if monto <= 0:
            raise ValueError("el monto a retirar debe ser positivo")
        if monto > self.saldo:
            raise ValueError("fondos insuficientes")
        self.saldo -= monto
```

`raise ValueError(...)` es la forma en Python de "lanzar un error" del pseudocódigo: interrumpe la ejecución inmediatamente con un mensaje que explica qué salió mal, en vez de dejar que el programa siga con un saldo negativo.

Pruebas con pytest

```
# test_cuenta.py
import pytest
from cuenta import CuentaBancaria

def test_deposito_incrementa_el_saldo():
    cuenta = CuentaBancaria("Ana", saldo_inicial=100)
    cuenta.depositar(50)
    assert cuenta.saldo == 150

def test_retiro_valido_reduce_el_saldo():
    cuenta = CuentaBancaria("Ana", saldo_inicial=100)
    cuenta.retirar(30)
    assert cuenta.saldo == 70

def test_retiro_sin_fondos_lanza_error():
    cuenta = CuentaBancaria("Ana", saldo_inicial=100)
    with pytest.raises(ValueError):
        cuenta.retirar(500) # pide retirar más de lo que hay: debe fallar
```

`pytest.raises(ValueError)` es la forma de probar que un error **debía** ocurrir: el bloque `with` espera que el código de adentro lance un `ValueError`; si lo lanza, la prueba pasa; si el código termina sin lanzar ningún error, la prueba falla — porque significaría que se permitió un retiro sin fondos suficientes.

Integrando todo: el flujo de trabajo completo

```
git init
git add cuenta.py test_cuenta.py
git commit -m "Agrega CuentaBancaria con depósito/retiro y sus pruebas"
pytest # ejecuta las 3 pruebas y reporta: 3 passed
```

Este es, en miniatura, el mismo flujo que se usa en proyectos reales: escribir la clase, escribir las pruebas que verifican su comportamiento, y guardar ambas cosas juntas en el mismo commit — porque un cambio de comportamiento sin su prueba correspondiente, o una prueba sin el código que la haga pasar, es un commit incompleto.

8. Notas de vigencia técnica (2026)

Testing: `unittest` viene incluido en la librería estándar de Python desde siempre y sigue siendo válido, pero **pytest** es hoy el estándar de facto en la industria: sintaxis más simple (funciones sueltas con `assert`, en vez de clases que heredan de `unittest.TestCase`), mejores mensajes de error y un ecosistema de **plugins** mucho más grande. Para proyectos nuevos, se recomienda empezar directamente con `pytest`.

Git: los comandos de esta semana (`init`, `add`, `commit`, `status`, `log`) son el núcleo de Git desde su creación en 2005 y no han cambiado. Lo que sí es cada vez más común es que estos mismos comandos se combinen con plataformas como GitHub, y que las pruebas de `pytest` se ejecuten automáticamente en cada `git push` mediante herramientas de integración continua (CI) — ese flujo completo (código + pruebas + ejecución automática) se retoma en el Módulo 6 en adelante conforme los proyectos del curso crecen.

POO: para clases simples que solo agrupan datos con poco o ningún comportamiento propio, Python moderno ofrece el decorador `@dataclass` (módulo `dataclasses`, disponible desde Python 3.7), que genera automáticamente el `__init__` a partir de los atributos declarados. Esta semana se enseña `__init__` escrito a mano porque es la base que explica **qué** hace `@dataclass` por debajo — una vez entendido esto, usar `@dataclass` en clases de solo-datos es una simplificación válida, no un concepto distinto.

9. Errores comunes de principiante

- Olvidar `self` como primer parámetro al definir un método (`def depositar(monto):` en vez de `def depositar(self, monto):`) — Python lanza un `TypeError` confuso sobre "número de argumentos", porque intenta pasar el objeto como si fuera `monto`.
- Usar una lista o diccionario como **atributo de clase** cuando en realidad se necesita un atributo de instancia — todos los objetos terminan compartiendo (y modificando) la misma lista sin darse cuenta (ver sección 3).
- Ejecutar `git commit` sin haber hecho `git add` antes (o sin agregar el archivo nuevo) — el commit se guarda, pero sin los cambios esperados, porque solo se guarda lo que está en el área de preparación.
- Hacer un único commit gigante con mensaje `"cambios"` al final del día, en vez de commits pequeños y descriptivos a medida que se completa cada parte — dificulta entender el historial y volver atrás a un punto específico si algo se rompe.

- Escribir pruebas que no verifican nada real, como `assert True` o `assert resultado == resultado` — pasan siempre, dan una falsa sensación de seguridad, y no detectan ningún error real en el código.

10. Ejercicio propuesto

Crea una clase `Rectangulo` con atributos de instancia `ancho` y `alto`, un atributo de clase `figuras_creadas` que cuente cuántos rectángulos se han creado en total, y dos métodos: `area()` (que devuelva `ancho * alto`) y `perimetro()` (que devuelva `2 * (ancho + alto)`).

Luego:

1. Inicializa un repositorio Git en una carpeta nueva y haz un primer commit con el archivo de la clase.
2. Escribe al menos dos pruebas con pytest: una que verifique `area()` con valores conocidos, y otra que verifique `perimetro()` con valores conocidos.
3. Haz un segundo commit que agregue el archivo de pruebas.

Pista: para probar `figuras_creadas`, crea dos o más objetos `Rectangulo` dentro de una misma prueba y verifica con `assert` que el contador de clase refleja cuántos se crearon — igual que se hizo con `Estudiante.total_estudiantes` en la sección 3.

11. Autoevaluación

1. ¿Qué representa `self` dentro de un método de una clase? — El objeto sobre el que se está llamando el método en ese momento; Python lo pasa automáticamente, por eso no se incluye al llamar al método, solo al definirlo.
2. ¿Cuál es la diferencia entre un atributo de instancia y un atributo de clase? — El de instancia pertenece a un objeto en particular (definido dentro de `__init__` con `self.`); el de clase se define directamente en la clase y su valor es compartido por todas las instancias.
3. ¿Por qué usar una lista como atributo de clase (en vez de atributo de instancia) suele ser un error? — Porque al ser un único valor compartido, todos los objetos terminarían leyendo y modificando la misma lista, en vez de tener cada uno la suya.
4. ¿Cuál es la diferencia entre `git add` y `git commit`? — `git add` marca los cambios que van a formar parte del próximo punto de guardado (área de preparación); `git commit` recién los guarda de forma permanente en el historial, con un mensaje.

5. ¿Para qué sirve un archivo `.gitignore`? — Para decirle a Git qué archivos no debe rastrear nunca, típicamente archivos generados automáticamente (como `__pycache__ /`) que no tiene sentido guardar en el historial.
6. ¿Qué hace `assert condicion` cuando `condicion` es falsa? — Lanza un error inmediatamente, interrumpiendo la ejecución; cuando es verdadera, no hace nada visible.
7. ¿Cómo identifica pytest automáticamente qué funciones son pruebas? — Busca archivos cuyo nombre empiece con `test_` y, dentro de ellos, funciones cuyo nombre también empiece con `test_`.
8. V/F: "Una prueba como `assert True` es útil porque siempre pasa, confirmando que el código funciona." — Falso; una prueba que siempre pasa sin verificar ningún valor real del código no detecta ningún error y solo da una falsa sensación de seguridad.
9. Explica con tus propias palabras, sin jerga técnica, por qué un equipo que entrena modelos de Machine Learning necesita Git además de las pruebas automatizadas. — Porque sin Git es casi imposible saber qué versión exacta del código produjo un modelo entrenado en particular (dificultando reproducir o corregir resultados), y sin pruebas automatizadas un cambio en el código de preparación de datos puede arruinar un entrenamiento completo sin que nadie lo note hasta mucho después.

12. Verificación de dependencias

Este contenido depende directamente de la Semana 5 (razonar sobre la corrección de una función antes de organizarla dentro de una clase y de cubrirla con pruebas — el método `promedio()` de esta semana reutiliza la misma lógica de las semanas anteriores) y de la Semana 4 (listas y diccionarios, usados aquí como atributos de instancia como `self.notas`). No se introduce ningún concepto matemático nuevo: los ejemplos de `CuentaBancaria` y `Rectangulo` usan solo aritmética básica. Es prerequisite directo del Módulo 7 (matemáticas para ML I: álgebra lineal y cálculo esencial), donde los ejemplos empiezan a organizarse como pequeñas clases y funciones probadas, en vez de scripts sueltos.